Final Project Requirements, Questions and Architecture

ADS 5984 - Agentic AI in Practice

Ryan Webster

Nov 6, 2026

Final Project Definition

The project you are designing here is **your** final project — either one of the ideas on the **Final Project Specifications** page, or a project idea the instructor has approved. Read that page end to end (overview, required showcase sections, rubric, and the idea you picked) before you start this exercise.

Assignment Overview

Anyone can build an agentic system with Claude Code or Open Code. *Can you build one that is reliable, safe, cost effective and meets all the requirements every time it's run?* It's time to put on your AI architect hat and address LLM/RAG shortcomings with a good agentic AI design.

Rubric

This is a 500 point exercise. It's an assessment of how much you have learned up to this point. Please take it seriously. It's as important as the project output. You should complete this Quarto markdown file (.qmd) and turn it in along (as .qmd) with a quarto rendered PDF version (.pdf). Add `format: pdf` to the yaml header of this file and then run `quarto render yourfilename.qmd` to build the PDF.

Important Note

This work must be done 100% without assistance from any AI. I need to know that *YOU* have learned the material. I know that Claude knows it. **Any use of AI will be easily detected and will be treated as an Honor Code violation. Please take this seriously.**

Introduction

Unlike the mid-term, there is no single specification for everyone — so before you list requirements, tell me what you are building. Name the project you chose: one of the ideas on the Final Project Specifications page, **or** a project the instructor has approved (say so, and when it was approved).

Then write a few paragraphs (in prose) explaining what you are trying to accomplish. Who is the user? What problem do they have today, and what will they be able to do with your solution that they cannot do now? What is in scope and — just as important — what is out of scope? What does a successful run look like? Finally, make the case for why this problem **needs a multi-agent design** rather than a single prompt or a script.

The project I have chosen is **34.4.13 Sports: Upset Condition Profiler**.

In this project, I am aim to forecast potential upsets in NCAA Division I Men's Basketball utilizing a multi-agent system to research common preconditions of past upsets in order to determine which current matchups follow those same signals. One user of this project would be an analyst/reporter who covers the sport whose job it is to research, break down, and highlight upcoming matchups. Like with any other sport, basketball is a high variance game which introduces a substantial amount of noise and luck, which can result in the empirically *better* team to lose a specific game, even if they would win >50% of the time over the span of a thousand simulated games. Typically, 11-12 points is a well-supported estimate for the standard deviation of scoring-margin errors relative to expected margins, which shows that, oftentimes, the best team doesn't always win. The problem today for these analysts is that forecasting potential upsets is tricky, it is something that happens often, but deciding *which* games are the actual upsets is difficult.

Betting markets are also quite accurate, with mainstream sources having an MAE of ~8 points when predicting final scoring margin. So, having to go against the markets can be dangerous. With this tool, analysts can look through advanced metrics and statistics that modelers might use and look at other factors that people and models may overlook like, travel fatigue, revenge games, rest days between games, overlooking opponent for the marquee matchup in their next game, etc. Researching these things isn't impossible for a team of analysts to do, however, but there are typically dozens of games a day

with the addition of all of the other responsibilities the analyst has. With this project, it will streamline the research process by providing the user with its upset predictions and justifications from the list of common factors of past upsets.

It's also important to note what the project can and cannot do or answer. Historical upset research, precondition extraction, upcoming matchup monitoring, upset risk scoring, multi-agent orchestration, and explainable recommendations are all features and functionalities that **are** in-scope for this project. On the contrary, guaranteed score predictions and scoring margin, real-time in-game analysis, automated sports betting, functionality outside of NCAA Division I Men's Basketball, and player-level forecasting are all features that are **not** in-scope. A successful run for this project would be an autonomous system in which the "upset scorer" agent takes the compiled research on upsets from the researcher and precondition-extraction agent and determine which games on that day are on upset-watch, ranked 1 to n number of games based on its upset score along with its explanations. These results would then be presented in the final deliverable of a Streamlit UI.

This project could, in theory, be done in a single-agent system or script, but would have many limitations and ultimately would provide non-satisfactory results. Using a single agent, the context would overflow from all of (and mostly irrelevant) information from all of the different tasks, which would degrade performance. Additionally, we wouldn't be able to parallelize the 'upset-ranking' portion of the system, which would bottleneck this project severely, given that most days there are usually dozens of games. Also, we would lose out on the ability to easier debug and audit.

This project benefits from a multi-agent design due to the specialized tasks required for each of the agents, the want for validation, and the ability to parallelize work during the 'upset-scorer' phase. Additionally, allowing agents to have specialized tasks should keep their context shorter and more focused, along with the ability for easier debugging and auditing. More specifically, the "upset-scorer" agent doesn't need the context of the "researcher" agent and it would most likely degrade performance. A single agent would make these needs more difficult, especially in validation, overload, and not being able to parallelize the 'upset-scorer' phase.

Requirements

List the requirements for **your** project, based on the Final Project Specifications (the overview, required showcase sections, rubric, and the description of the idea you chose — or your instructor-approved proposal) and on the design you are proposing in your Introduction. List as many as you can (20 is not meaningful here). This is a critical step! Any person or LLM should be able to fully understand **all** requirements for the application from this list.

Number every requirement — the later sections refer back to them by number.

1. The solution should provide a usable and responsive UI using Python's Streamlit that exposes the solution and shows agent activity.
2. The solution should let the user sort/filter today's games by certain fields or select certain games for analysis by the `Monitor` and `Upset-Scorer` agents if they so choose so.
3. The solution should let the user rerun the process as many times as they wish with any combination of games.
4. The solution should utilize an Orchrestrator-worker architecture for the historical research ingestion phase of the project. Let the `Researcher-Orchrestrator` utilize different workers to scrape structured statistical data and other contextual information and preconditions about the game.
5. The solution should let workers for the `Researcher-Orchrestator` use MCP servers like `Brave Search`, `Fetch`, and `Deep Research` along with a sports API for statistical data.
6. The solution should parallize the work done from the `Researcher-Orchrestator` agent in order to reduce latency on this long and expensive task.
7. The solution should let the `Researcher-Orchrestrator` aggregate its findings from its workers by joining on `game_id`. It should also enforce Pydantic's JSON schema and validate JSON results.
8. The solution should store the results into either a SQLite database or ChromaDB RAG system, depending on the information being stored (i.e. structured game statistics data should go into the SQLite database, cited documents should go into the RAG). It should also rerank using a decision model.
9. The solution should utilize an `Extractor` agent to take research given from the `Deep-Researcher` agent and determine what common preconditions are actual useful signal in determining potential upsets. It should utilize deterministic statistical validation code to determine if the preconditions are actually statistically significant.
10. The solution should utilize a batch data pipeline that extracts scheduled games for that day using the ESPN API based on what games the user chooses. The API should only return Division I versus Division I matchups, along with team, location, team and player statistics, and betting odds to determine who is the favorite.
11. The solution should retrieve the daily matchups for the Monitor agent and Streamlit UI for displaying today's slate of games using the ESPN API.

12. The solution should utilize a `Monitor` agent, which will use the research done by the `Deep-Researcher` and `Extractor` agents, along with that game's statistics and preconditions, to flag which games are considered upset potential.
13. The `Monitor` agent should provide rational in its choice of which games are considered to be a potential upset.
14. The solution should then utilize a `Upset-Scorer` agent, which will take the output from the `Monitor` agent, including its rational, and then utilize VT ARC's d3-mini decision model with a type of score to rank which upsets are most prone to happen.
15. The solution should let the `Deep-Researcher` agent output its final work in a JSON database, in which the `Extractor` can do its work, and store its results in a separate table.
16. The solution should provide well-structured code with descriptive comments and run instructions.
17. The solution should let the `Monitor` agent run in parallel with its orchrestrator.
18. The solution should provide the cited rational from the `Monitor` agent along with the score from the `Upset-Scorer`'s decision model.
19. The solution should be highly reliable. Running the ranker on the same matchups should generate relatively consistent results. If the results are wildly different, the solution is a complete failure and will never be accepted.
20. The solution should utilize a `.env`, while keeping it separate from Git and using `.env.example` as a boilerplate for the user.
21. The solution should conform to a standardized JSON output using Pydantic to prevent broken handoffs.
22. The solution should properly handle API failures, LLM timeouts, and MCP failures without crashing the whole application.
23. The solution should cache matchups that already have been analyzed.
24. The solution should not have any Information on a game being researched by the `Deep-Researcher` agent known *after* tip-off to prevent data leakage. Use timestamps of when information was known so that it can be filtered out by the time of tip-off.
25. The solution should also validate the upset probabilities produced by the `Upset-Scorer` agent, using seasons 2021-2024 as training data, the 2024-2025 season as the validation set, and 2025-2026 as the testing set. It's important to know that the results being predicted are meaningful.

LLM / RAG Issues

Referring back to the lectures on LLM and RAG — and, now that you have seen them, on multi-agent systems and anti-patterns — what are the specific issues that you must overcome with good agentic AI design/architecture/patterns **to meet the requirements**? **Make sure you point back to a specific requirement**. Don't solve the issue yet. Just identify it. You should have a fair number of these (between 5 and 20).

1. For requirement 4, the specific LLM issue that needs to be addressed are hallucinations. Hallucinations can provide confident, but inaccurate responses for a multitude of reasons (i.e. lost-in-the-middle context, context overflow, unclear instructions, etc.). This can especially be a cause for concern for the `Context-Research` worker who has to read many different documents of many different games. If allowed, the evidence used for the `Monitor` agent will confidently flag games as an upset using incorrect assumptions about preconditions for upsets.
2. For requirement 5, the specific LLM issue that needs to be addressed is prompt injection. Prompt injections allow vulnerabilities or faulty information which can influence the LLM into providing incorrect answers. This can be a cause for concern for both the workers under the `Researcher-Orchrestrator` who will read tons of research material from the open web. If allowed, the evidence used for the `Monitor` agent will confidently flag games as an upset using incorrect assumptions about preconditions for upsets.
3. For requirement 8, the specific RAG issue is chunking. There are many different approaches to chunk the documents that will be stored in ChromaDB. Choosing the wrong chunking strategy can be an issue, however, where chunking too coarsely will retrieve mostly irrelevant material and blowing up context, while chunking too finely and we'll fragment ideas across multiple chunks. In this example, improperly chunking, especially across different documents from different games, can provide a poor output.
4. For requirement 9, the specific LLM issue is context-window limitations. This issue essentially makes agents hallucinate and perform poorly due to all of the information provided across many different games. These documents will be read from the RAG for the `Extractor` agent for its statistical validation process and may clutter its context, causing it to overflow and degrade performance.
5. For requirement 19, the specific LLM issue that needs to be addressed is inconsistency and nondeterminism. This issue allows LLMs to output a wide variety of results, essentially making them useless. Ideally, we would like the `Monitor` and `Upset-Scorer` agents alike to provide mostly consistent results for the same slate of games.
6. For requirement 21, the specific LLM issue is producing a JSON file following a certain schema. Whenever there is a tool call, or in this case storing structured data into the SQLite database, it's imperative that the JSON output from an LLM follows the

required format. Otherwise, the application may crash if the expected JSON schema isn't followed.

7. For requirement 24, the specific LLM issue that needs to be addressed is data leakage. This issue allows agents or models to train or act on material not intended to be acted upon. By doing so, our agents will make conclusions from the data that it shouldn't have known. In this case, we want the `Context-Research` worker to **only** retrieve information that was known **before** the beginning of the game. Even if we tell the agent in its constitutional prompt, it may inadvertently acquire material from after the tip-off. For example, a star player for a team may've been injured, but wasn't known until afterwards. Essentially, the `Monitor` agent would be making predictions based off material not known to the public.

Basic Architecture

Write up a couple of paragraphs (in prose) on how you plan to design the solution. What will be the general approach. Don't solve the issues yet… just general approach. What are the major pieces, how do they connect, etc.?

The solution will be architected as a multi-agentic system using an Orchestration-Worker combined with sequential/pipeline stages. In the deep research portion, the `Research-Orchrestrator` agent will take two different workers, `Historical-Stats` and `Context-Research`, and decompose the main tasks into the self-contained subtasks. The `Historical-Stats` worker's main goal is to compile box-scores of researched games using a College Basketball API, while the `Context-Research` worker will compile news, pregame reports, and other preconditions about the game. These workers will utilize a MCP server with `Brave Search`, `Fetch`, and `Deep Reserach` to help with the search. The `Research-Orchrestrator` agent will then aggregate the results, joining by `game_id` and then enforce Pydantic's schemas. This will be a one time process in which the user themselves cannot execute, as its expected to take a long time to complete.

Afterwards, all structured data will be stored in a SQLite database as a JSON along with any cited unstructured documents in a RAG system using ChromaDB and embedded using qwen3-embedding:8b and the d3-mini decision model as the reranker. Then, the `Extractor` agent will use a variety of statistical validation tools and methods to determine what preconditions have statistically significant effects on upsets, with the final output being a Pydantic enforced schema saved into a separate SQLite table. Then, the `Monitor` agent will read that day's matchups from an ESPN API and determine if any of the predetermined statistically significant preconditions occur in that matchup, if so, that matchup is then flagged and an explanation will be handed off to the proceeding `Upset-Scorer` agent. Additionally, the `Monitor` agent step is all ran in parallel, as it's an embarrassingly parallel task. Once the work is done and aggregated back into the `Monitor-Orchrestrator` agent,

the `Upset-Scorer` will take a look at the flagged games and assign a probability of an upset using the d3-mini decision model, and return the final results to the Streamlit UI.

The user will be able to select any or all games to be evaluated from that day's slate of games from a list which can be filtered or sorted. By default, all games will be selected, but allowing the user to selected their preferred game will allow latency to drop. The user will then be able to click an `Execute Search` button, starting the Monitoring process with attached progress on what agent is doing what

Off to the side, we will also want to validate our results using the upset probability outputs from previous seasons as training data, to determine how will this system actually performs. Metrics like log loss and Briar score will be reported along with a calibration plot. Without this, the outputs from the `Upset-Scorer` would be mostly meaningless when the product is first shipped.

You should end your `Basic Architecture` with the level of agency you think is **earned** by this solution architecture. Go back to the beginning of the course for the various levels. **Make sure you give a quick summary of why it is earned.**

The level of agency earned by this solution architecture is a **multi-agent design** with a Orchestration-worker workflow with sequential/pipeline steps. This is due to the need of parallelism in the research and monitoring phase, along with the need to sequentially handoff outputs from the `Extractor`, `Monitor`, and `Upset-Scorer` agents. This is different from the Supervisor-Router design that only allows work to be done fully sequentially in an essentially a ReAct loop. Also, there is no need for the `Context-Research` and `Historical-Stats` workers to communicate with each other and there is also a given plan up-front in our case. Compared to a single agent design, there would simply be too much context in its context-window to produce a quality output, especially with having unrelated information congesting its context window.

25 Bonus points for including a UML sequence diagram using a mermaid diagram. For this syntax (and only this syntax), I will allow LLM usage** but you need to prompt the LLM with the sequence diagram flow, actors and steps. Include your prompt here.**

Create a UML sequence diagram using Mermaid for my multi-agent, NCAA Division I Men's Basketball Upset Condition Profiler which utilizes a combination of Orchrestrator-Worker architecture along with sequential pipeline phases. You will create three different diagrams representing the different phases of the project: historical research ingestion, statistical precondition extraction, and daily matchup monitoring and upset scoring. Make sure you include sequence diagram flow, actors, and steps.

Actors

Include: User, Streamlit UI, Research-Orchestrator, Historical-Stats agent, Context-Research agent, College Basketball API, MCP tools (Brave Search, Fetch, Deep Research), Pydantic validator, SQLite database, ChromaDB, embedding model (qwen3-embedding:8b), d3-mini reranker/decision model, Extractor agent, statistical validation tools, ESPN API, Monitor-Orchestrator, Monitor workers, and Upset-Scorer agent.

Historical Research Ingestion

- This phase will be a one-off process, not executed by the user.
- The `Research-Orchrestrator` will have two workers, `Historical-Stats` and `Context-Research`. The Research Orchrestrator will decompose the work into the two workers.
- There will be a loop where the orchrestrator will send a batch of games (number to be determined) to the workers. Make sure to use the loop keyword on the graphic.
- In the loop, the orchrestrator will provide the game ids. The Historical Stats worker will request box scores and statistics from the College Basketball API and return the historical, structured game records. This process can be done in parallel, use the par keyword.
- For the Context Researcher, use the assigned game ids, use the Brave, Fetch, and Deep Research MCP servers to gather reports, news, etc. Return the documents to the Orchrestrator. This is also done in par.
- After the loop finishes, the orchrestrator will aggregate work and validate to the Pydantic Validator.
- If successful (use alt keyword), have the orchrestrator send the validated JSON to the SQLite db, confirm it, then chunk source documents and then embed it in the qwen3-embedding:8b model, then store in ChromaDB, send confirmation the the orchrestrator when completed.
- If failed, have the Pydantic Validator return the errors to the orchrestrator, have it log errors and quarantine failed records.
- End loop and alt

Statistical Precondition Extractor

- Like with the first diagram, this will be done one time, not initiated by the user.
- The Extractor Agent should retrieve historical game records from SQLite along with the candidate precondition.
- In a loop the Extractor Agent should embed evidence (for each candidate precondition) for the search query and retrieve related historical evidence from ChromaDB. Use the decision model to re-rank evidence and return to the Extractor
- Then, send all statistical and document evidence to our statistical validation tools which will estimate effect sizes, confidence intervals, evaluate statistical significance, and test out-of-sample predictive validity.
- Return results back to Extractor.
- The extractor will the accept signals with statistics and citations, then compile all results. Send to Pydantic validator to validate schema
- In alt block, if accepted, store preconditions and statistical evidence in a separate validated preconditions table in SQLite. if failed, attempt to correct then invalidate bad entries.

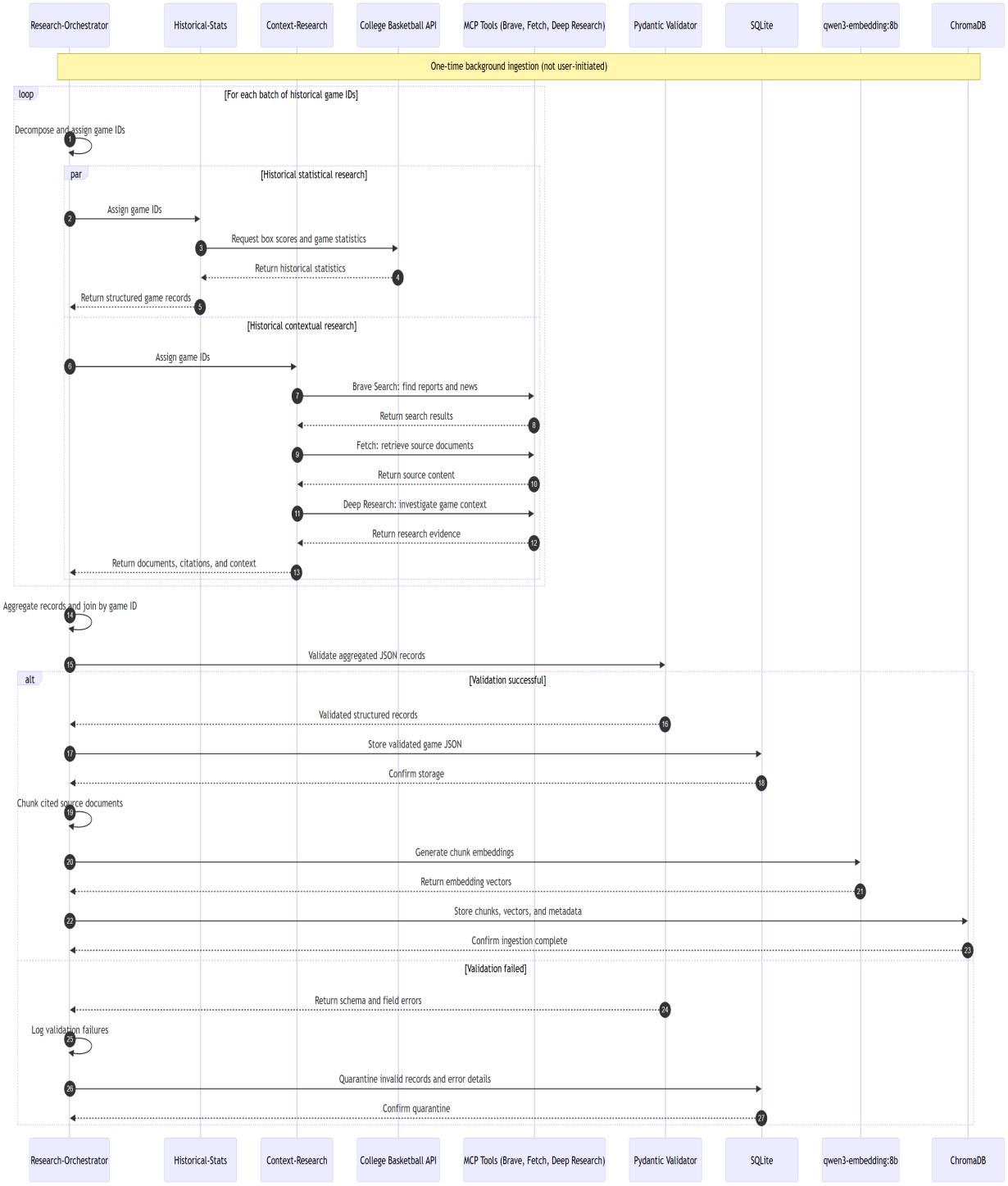
Daily Matchup Monitoring + Upset Scorer

- The user will open up the Condition Profiler app via the Streamlit UI.
- The UI will then retrieve all D1 vs. D1 matchups for that day from the ESPN API and display sortable and filterable matchups with basic statistics and game information.
- In a large loop, the user will be able to select any combination of games they want to further examine, or all of them by default
- After the user clicks search, there will be an agent progress tracker tracking its progress.
- The UI will then send game ids to the Monitor Orchrestrator agent which will retrieve validated preconditions from the SQLite db. Then The Monitor Orchrestrator will divvy out the work amongst two different worker agents where each one, in parallel, retrive current matchup data from the ESPN API, and research pregame conditions and context, check with the validated precondtions, and flag games with evidence if flagged. There will be two worker agents for the monitor, so basically have two par blocks for each. Make sure the UI is also aware of its progress.

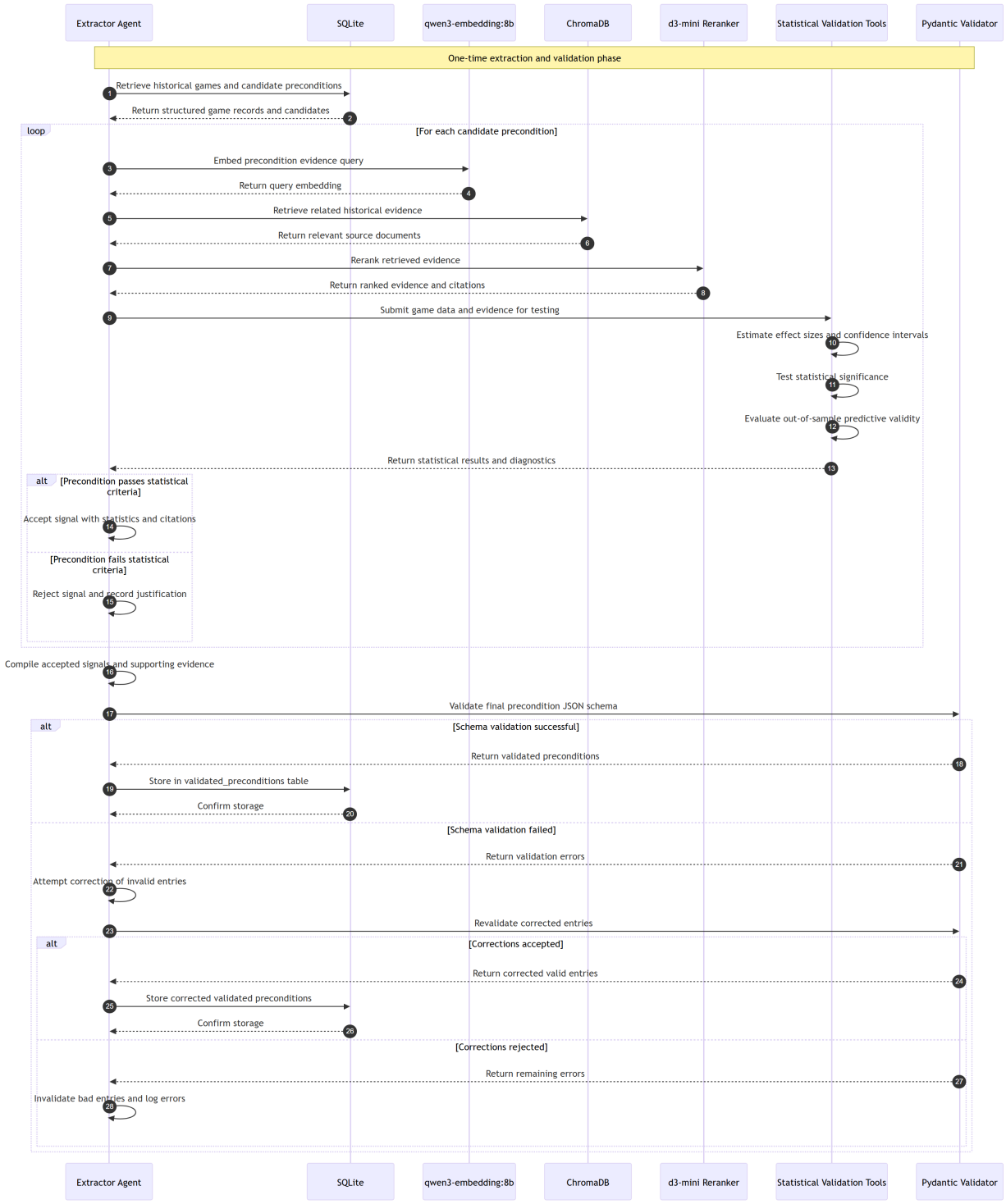
- Once the work is complete, have the orchrestrator aggregate results.
- Have an alt block for if games are found or not. If found, send to upset scorer agent which will then use the d3 decision model which will evaluate and return probabilities of an upset. Return results to the monitor orchrestrator which will return results to the UI. If no games worth mentioning were found from the original monitoring stage, return that there were no upsets found based on the preconditions.
- Display results to UI.

GPT-6.1-Sol High

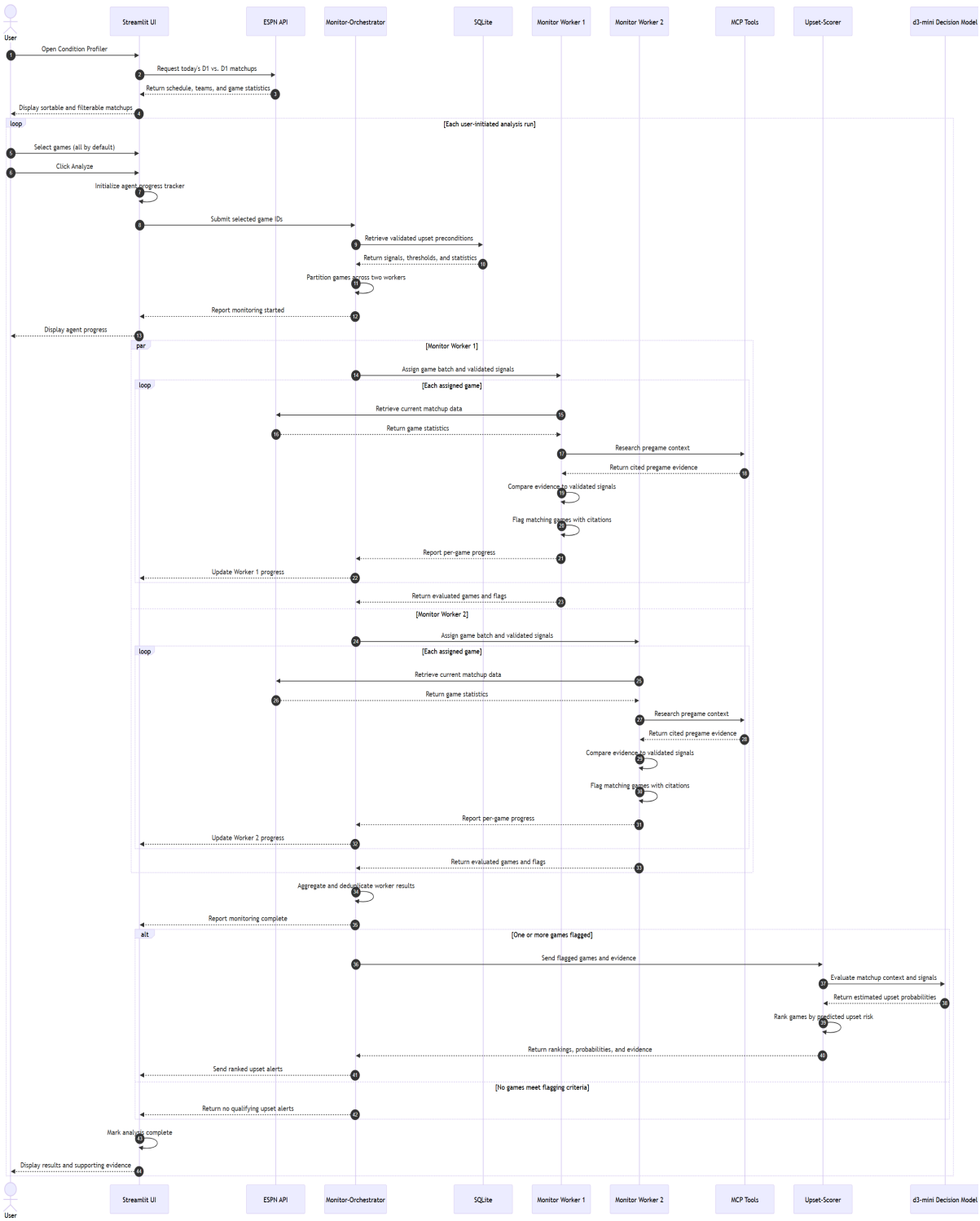
Historical Research Ingestion



Statistical Precondition Extraction



Daily Matchup Monitoring + Upset Ranking



Solution Mitigation

Now for each of the LLM/RAG issues, you should give specific design patterns that we learned in class to address them. I need a short explanation of why your solution is the right one. **Make sure you refer back to the numbered issue from the previous section.**

1. For issue 1, the design pattern that addresses this issue is limiting the amount of information the worker agents can use at a time. In my design, the Orchrestrator will send out batches of game requests at a time for the workers to work on. In doing so, this can limit the amount of content and allow more focused content in their context-window to mitigate hallucinations. Also, we need to account for lost-in-the-middle context as well, put more relevant content in the front or back to produce better results for the `Extraction` and `Monitor` agents. These are the best approaches as letting the workers in the research phase collect documents from different games, especially the same matchups on different dates, to pollute the context window and lead to hallucinations.
2. For issue 2, the design pattern that addresses this issue is manually whitelisting a set of websites the worker agents can use to extract information. Essentially, only gather data from reliable websites like ESPN.com, Sports-Reference.com, etc., along with local media to prevent any direct or indirect prompt injections that may throw off the final outputs. The reason why this is the best approach is because its the simplest, but most effective way to ensure the content that is feeding into my workers is legitimate and with no malicious intent as opposed to just telling the LLM "Make sure the website is legitimate".
3. For issue 3, the design pattern that addresses this issue is implementing recursive chunking within each document. The reason why I think this is the best choice is because we want to prevent any data leaking from one document into another while also respecting the natural document structure and producing semantically cleaner chunks. Other methods like semantic chunking would explode latency and fixed-size chunking where we would have to tune the two N and M parameters.
4. For issue 4, the design pattern that addresses this issue is allowing the `Extractor` agent to sequentially validate each precondition at once by taking all of the necessary material from the RAG/SQLite database related to that precondition as opposed to gathering all of the information at once. The reason why I think this is the best approach is because, by doing so, it keeps the context-window focused with less and more related material as opposed to retrieving it all at once and now we have a bloated context-window with less focused material.
5. For issue 5, the design pattern that addresses this issue is using a Jev-like decision model to take the outputs from the `Monitor` agent and its explanations and let the `Upset-Scorer` agent use a decision model to score the most likely upsets. Decision

models, like Jev, have shown to provide more consistent answers as it scores candidates based on a probability distribution as opposed to sampling output tokens. That is why I think it's the best choice instead of using a traditional LLM, which would more likely be less consistent and add latency.

6. For issue 6, the design pattern that addresses this issue is implementing a validation check for the output JSON structure. Using Pydantic is a good start, but even then, the output may still give invalid JSON syntax, incorrect data types, invalid values, etc. So, we should use the Orchrestrators to help validate the responses from its workers. The reason why I think this is the best choice is because we are already using the Orchrestrator as the centralized coordination amongst its workers, making it an obvious choice for it to validate the JSON responses.
7. For issue 7, the design pattern that addresses this issue is implementing timestamps from when the article is written and filtering out those who exceed the timestamp of when the game was tipped-off. For example, the `Context-Researcher` worker can extract the date of when the article was written and compare it to when the game started. If it happened before, keep it for the research and if not, omit it. The reason why I think this is the best choice is because this can be done by writing a simple script as opposed to having the LLM guess if the article was written beforehand.

Final Thoughts

Just like the mid-term, this exercise is something you do before invoking an agentic harness. This is where you add value and earn your place in the future AI ecosystem. Next steps we will review this architecture with a frontier level model and refine it. Then we work with an agentic harness to build it!

For this project-planning phase, I had to delve deeper into the many different types of multi-agent designs in order to determine which approach worked best for my project. Having to weigh the pros and cons of each helped me learn more about each of the different centralized and decentralized approaches. Additionally, it made me expand on the feedback from the midterm project for the possible LLM/RAG issues along with its mitigations. Also, this assignment forced me to think and research on the best solution to mitigate hallucinations, context rot, latency, etc. on multi-agent designs not present in the midterm project.